

动态规划

《算法设计与分析》

信息科学与技术学院 张德富



05



装配线调度问题



动态规划

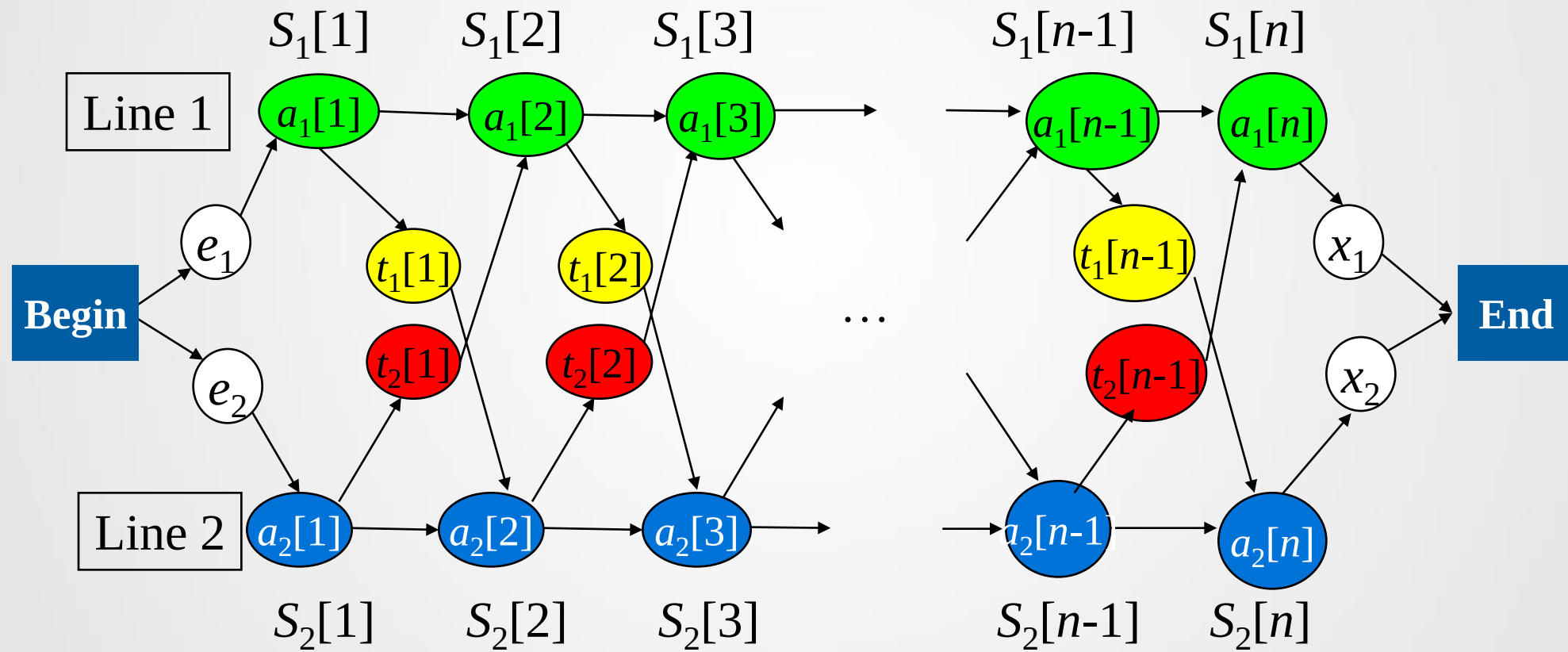
- **动态规划算法常用来求解最优化问题，设计一个动态规划算法通常有以下四个步骤**
 - 1. 找出最优解的结构；**
 - 2. 递归定义一个最优解的值；**
 - 3. 以自底向上的方式（从最简单问题开始入手）计算出最优解的值；**
 - 4. 根据计算最优解的值的消息，构造一个最优解。**



装配线调度问题

问题描述

- 有两条装配线，每一条装配线上有 n 个装配点，将装配线 i 上的第 j 个装配点记为 $S_i[j]$ ，设在该装配点的装配时间为 $a_i[j]$ 。假设要装配一辆汽车，将汽车底盘从进厂点送入装配线 i (1或2)，需要时间 e_i 。在装配点装配后，如果汽车传送到同一条装配线的装配点进行装配，则传送不需要时间。如果汽车从装配点传送到另一条装配线进行装配，则需要传送时间 $t_i[j]$ 。汽车在装配点 $S_i[j]$ 装配后，将汽车成品从装配线上取下来，需要花费时间 x_i 。装配线调度问题是如何确定每一个装配点的装配需要在哪条线上进行，使得当汽车成品出来时，花费的总时间最少。值得注意的是两条装配线的第个装配点都装配同样的汽车部件，只是装配效率不一样。



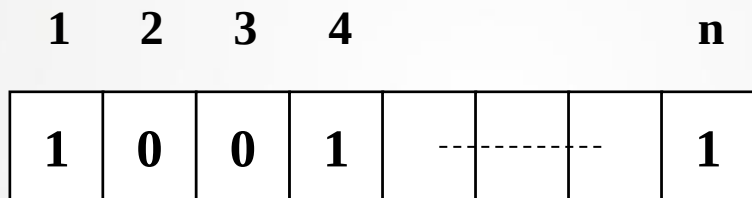


一个办法

■ 枚举法

- 在选择装配点时，枚举所有可能的选择
- 计算每条装配路线所需要的时间，然后选择最好的

■ 缺点：



0 if choosing line 2
at step j ($= 3$)

1 if choosing line 1
at step j ($= n$)

- 共有 2^n 条装配路线
- 当 n 很大时，实际上变得不可行



1. 最优解的结构

- 考虑从进厂到装配点 $S_1[j]$ 可能的最快路线
 - 如果 $j = 1$: 确定花多长时间到 $S_1[1]$
 - 如果 $j \geq 2$, 到达 $S_1[j]$ 有两个选择:
 - 过 $S_1[j-1]$, 然后到 $S_1[j]$
 - 过 $S_2[j-1]$, 然后到 $S_1[j]$



1. 最优解的结构

- 定理6.1:

假定最快装配路线过 $s_1[j-1]$ 然后到 $s_1[j]$, 则该路线中从进厂到装配点 $s_1[j-1]$ 的路线一定是最快的

- 如果最快装配路线过 $s_1[j-2]$, 则有同样的结论



1. 最优解的结构

- **一般性：**从进厂到装配点 $S_1[j]$ 的最短路线问题的最优解中一定包含了从进厂到装配点 $S_1[j-1]$ or $S_2[j-1]$ 的最短路线子问题的最优解。
- 这就是最优子结构性质,它是一个问题能够用动态规划求解的标志。
- 我们可以利用这个性质由子问题的最优解构造出问题的最优解。



2. 递归方法

- 令 $f_i[j]$ = 从进厂到装配点 $S_i[j]$ 的最快装配时间
- 令 f^* 表示问题的最优解, 则

$$f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$$

- 当 $j = 1$ 时

$$f_1[1] = e_1 + a_1[1]$$

$$f_2[1] = e_2 + a_2[1]$$

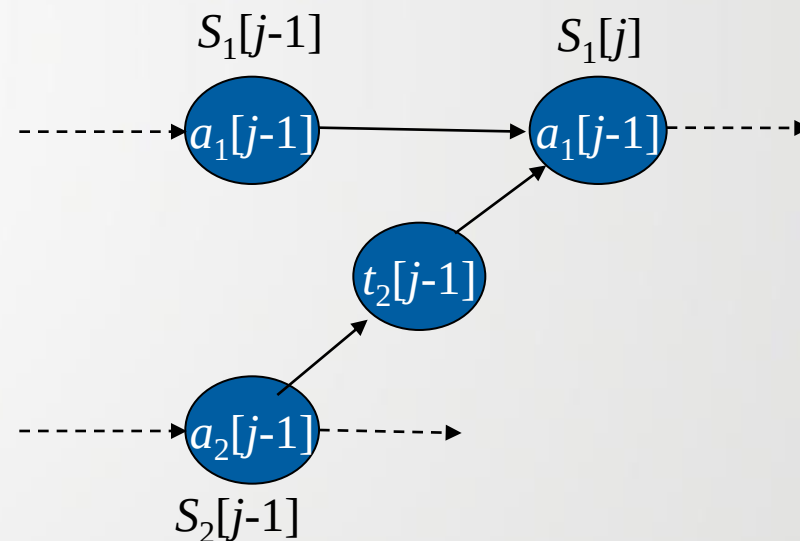


2. 递归方法

- 计算 $f_i[j]$ ($j = 2, 3, \dots, n$, and $i = 1, 2$)
- 从进厂到装配点 $S_1[j]$ 的最短路线, 只有两种情况:
 - 要么经过 $S_1[j-1]$ 然后直接到 $S_1[j]$, 即 $f_1[j] = f_1[j-1] + a_1[j]$
 - 要么经过 $S_2[j-1]$ 然后传送到装配线 $S_1[j]$, 此时有
$$f_1[j] = f_2[j-1] + t_2[j-1] + a_1[j]$$

因此, 可以选择最小的, 即

$$f_1[j] = \min(f_1[j-1] + a_1[j], f_2[j-1] + t_2[j-1] + a_1[j])$$





2. 递归方法

■ 可以得到递归方程

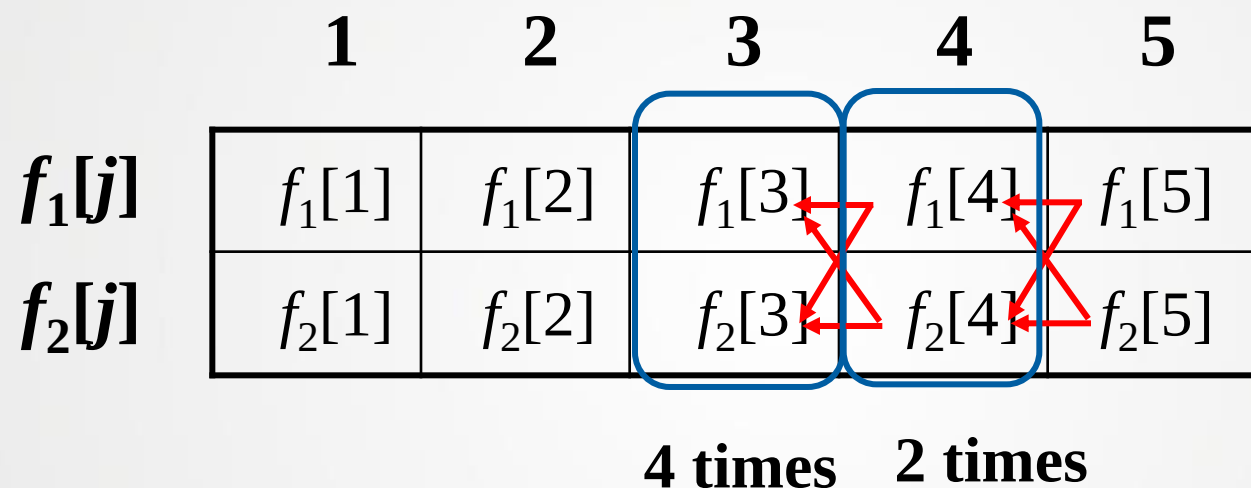
$$f_1[j] = \begin{cases} a_1[1] + e_1 & \text{if } j = 1 \\ \min(f_1[j-1] + a_1[j], f_2[j-1] + t_2[j-1] + a_1[j]) & \text{if } j > 1 \end{cases}$$

$$f_2[j] = \begin{cases} a_2[1] + e_2 & \text{if } j = 1 \\ \min(f_2[j-1] + a_2[j], f_1[j-1] + t_1[j-1] + a_2[j]) & \text{if } j > 1 \end{cases}$$



2. 递归方法

- 例如, $n=5$:



- 如果按照递归的方式至顶向下求解, 则计算量为指数级



3. 计算最优解

- 当 $j \geq 2$, $f_i[j]$ 的值只依赖 $f_1[j - 1]$ 和 $f_2[j - 1]$ 的值
- 计算 $f_i[j]$ 的值
 - 按照 j 增加的顺利

increasing j



	1	2	3	4	5
$f_1[j]$					
$f_2[j]$					

- 自底向上的方法
 - 首先找子问题的最优解
 - 利用子问题的解获得原问题的解



构造最优解

- 为构造最优解的值，我们需知每个站是在那条线上装配的，令
 - $l_i[j]$ 表示装配点 $S_i[j]$ ($j = 2, 3, \dots, n$) 的前一个装配点($j - 1$)的装配线号(1, 2)
 - l^* 表示出厂时，最快装配路线中装配点 n 所在的装配线号



	2	3	4	5
$l_1[j]$				
$l_2[j]$				



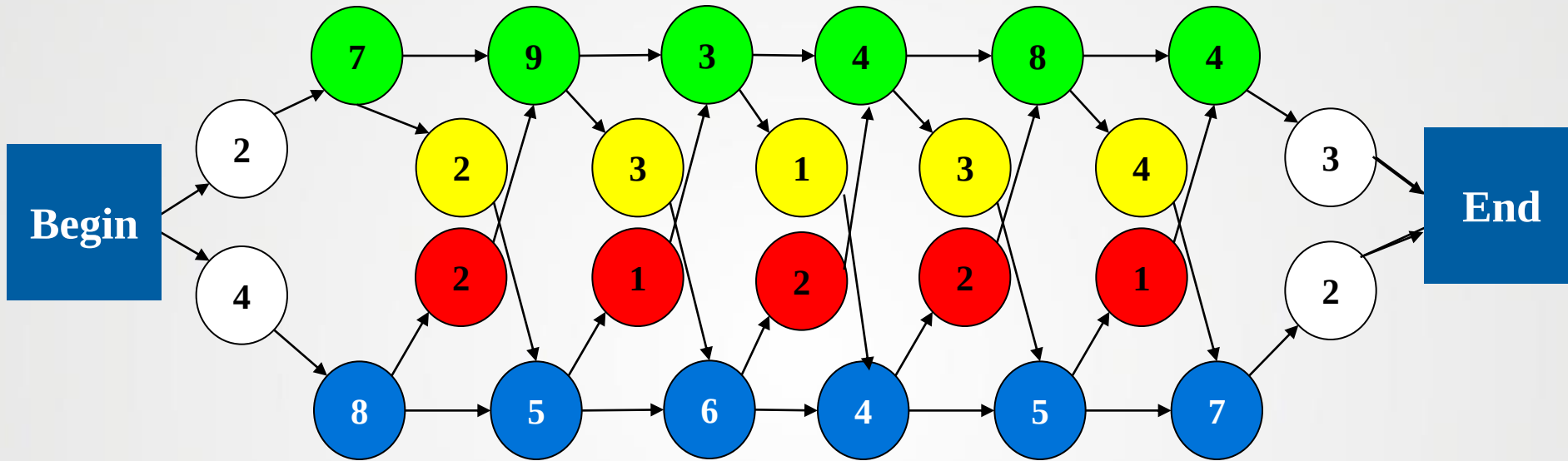
Step 3: 计算最快装配时间及路线

DPFastestWay(a, t, e, x, n)

```
1  $f_1[1] \leftarrow e_1 + a_1[1]; f_2[1] \leftarrow e_2 + a_2[1]$ 
2 for  $j \leftarrow 2$  to  $n$  do
3     if  $f_1[j-1] + a_1[j] \leq f_2[j-1] + t_2[j-1] + a_1[j]$  then
4          $f_1[j] \leftarrow f_1[j-1] + a_1[j]$ 
5          $l_1[j] \leftarrow 1$ 
6     else  $f_1[j] \leftarrow f_2[j-1] + t_2[j-1] + a_1[j]$ 
7          $l_1[j] \leftarrow 2$ 
8     if  $f_2[j-1] + a_2[j] \leq f_1[j-1] + t_1[j-1] + a_2[j]$  then
9          $f_2[j] \leftarrow f_2[j-1] + a_2[j]$ 
10         $l_2[j] \leftarrow 2$ 
11    else  $f_2[j] \leftarrow f_1[j-1] + t_1[j-1] + a_2[j]$  then
12         $l_2[j] \leftarrow 1$ 
13 if  $f_1[n] + x_1 \leq f_2[n] + x_2$  then
14      $f^* \leftarrow f_1[n] + x_1$ 
15      $l^* \leftarrow 1$ 
16 else  $f^* \leftarrow f_2[n] + x_2$ 
17      $l^* \leftarrow 2$ 
```

Running time: $\Theta(n)$

For example



j	1	2	3	4	5	6
$f_1[j]$	9	18	20	24	32	35
$f_2[j]$	12	16	22	25	30	37

$f^*=38$

j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$l^*=1$



4. 构造最优解

PrintStations(l, n)

1 $i \leftarrow l^*$

2 print “line ” i “, station ” n

3 **for** $j \leftarrow n$ **downto** 2 **do**

4 $i \leftarrow l_i[j]$

5 print “line ” i “, station ” $j - 1$

line 1, station 6

line 2, station 5

line 2, station 4

line 1, station 3

line 2, station 2

line 1, station 1

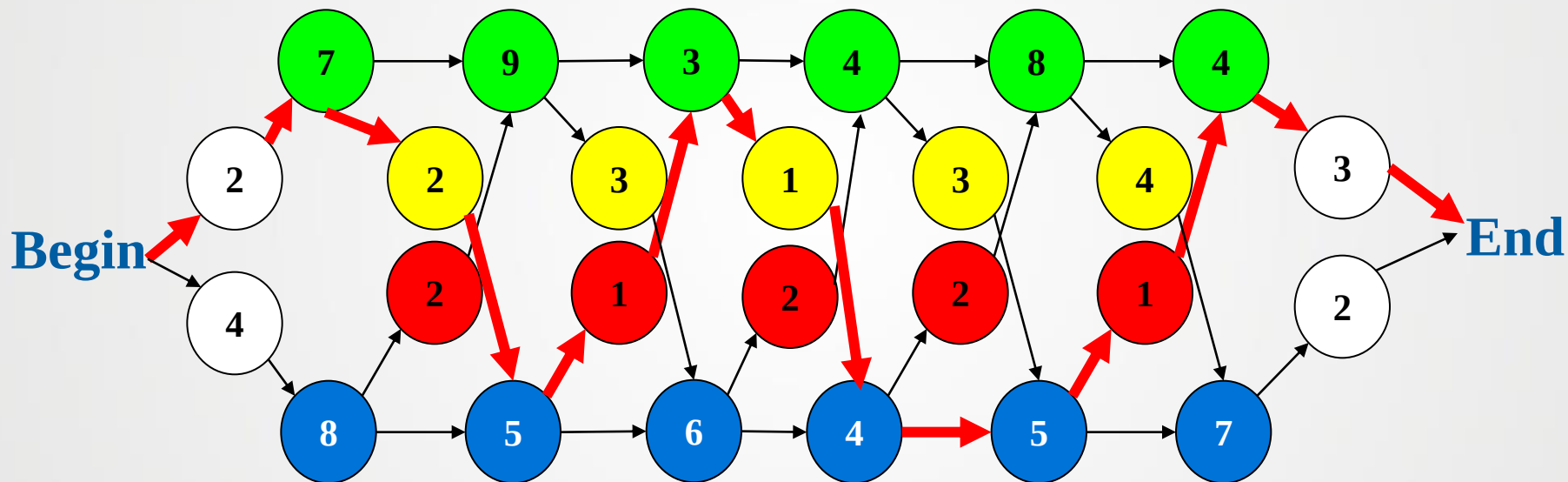
j	2	3	4	5	6
$l_1[j]$	1	2	1	1	2
$l_2[j]$	1	2	1	2	2

$l^* = 1$



4. 构造最优解

最快装配时间: $f^*=38$



红线箭头表示最快装配路线

谢谢大家